

Method Overriding – Bank (Dynamic Polymorphism)

Experiment No: 06 Date: 19/09/2026

■ Objective / Aim

To design and implement a Banking application in Java that demonstrates method overriding and dynamic (runtime) polymorphism, where a superclass reference invokes subclass-specific overridden behavior based on the actual object type at runtime.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

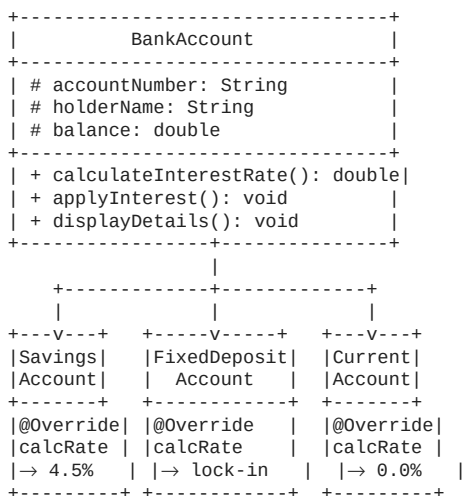
Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a base class **BankAccount** with attributes `accountNumber`, `holderName`, `balance`.
2. Define a method **`calculateInterestRate()`** in **BankAccount** that returns a default interest rate.
3. Define a method **`applyInterest()`** in **BankAccount** that adds interest to the balance using `calculateInterestRate()`.
4. Create a subclass **`SavingsAccount`** extending **BankAccount** that overrides `calculateInterestRate()` to return a higher fixed rate.
5. Create a subclass **`FixedDepositAccount`** extending **BankAccount** that overrides `calculateInterestRate()` to return a rate based on a lock-in period.
6. Create a subclass **`CurrentAccount`** extending **BankAccount** that overrides `calculateInterestRate()` to return zero interest.
7. In `main()`, create an array/list of **BankAccount** references pointing to different subclass objects.
8. Loop through the references and call `applyInterest()` on each – dynamic polymorphism ensures the JVM resolves the correct overridden `calculateInterestRate()` at runtime based on the actual object, not the reference type.
9. Display each account's details after interest is applied.

■ UML Class Diagram



Runtime dispatch: BankAccount ref → actual object decides which `calculateInterestRate()` runs (Dynamic / Runtime Polymorphism). `SavingsAccount`, `FixedDepositAccount`, and `CurrentAccount` all inherit from `BankAccount` and override `calculateInterestRate()`. When called through a `BankAccount` reference, the JVM performs dynamic method dispatch.

■ Source Code

```
// Base class
class BankAccount {
    protected String accountNumber;
    protected String holderName;
    protected double balance;

    BankAccount(String accountNumber, String holderName, double balance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
    }

    // Default interest rate - to be overridden
    public double calculateInterestRate() {
        return 2.0; // generic default rate (%)
    }

    // Applies interest using whichever calculateInterestRate() resolves at runtime
    public void applyInterest() {
        double rate = calculateInterestRate();
        double interest = balance * rate / 100;
        balance += interest;
        System.out.printf("Interest Rate Applied : %.2f%%\n", rate);
        System.out.printf("Interest Credited      : Rs.%.2f\n", interest);
    }

    public void displayDetails() {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Holder Name   : " + holderName);
        System.out.printf("Balance      : Rs.%.2f\n", balance);
    }
}

// Savings Account - overrides interest rate
class SavingsAccount extends BankAccount {
    SavingsAccount(String accountNumber, String holderName, double balance) {
        super(accountNumber, holderName, balance);
    }
    @Override
    public double calculateInterestRate() {
        return 4.5; // higher fixed rate for savings
    }
}

// Fixed Deposit Account - overrides interest rate based on lock-in period
class FixedDepositAccount extends BankAccount {
    private int lockInYears;
    FixedDepositAccount(String accountNumber, String holderName,
                        double balance, int lockInYears) {
        super(accountNumber, holderName, balance);
        this.lockInYears = lockInYears;
    }
    @Override
    public double calculateInterestRate() {
        if (lockInYears >= 5) return 7.5;
        else if (lockInYears >= 3) return 6.5;
        else return 5.5;
    }
}

// Current Account - overrides interest rate to zero
class CurrentAccount extends BankAccount {
    CurrentAccount(String accountNumber, String holderName, double balance) {
        super(accountNumber, holderName, balance);
    }
    @Override
    public double calculateInterestRate() {
        return 0.0; // current accounts earn no interest
    }
}

// Main class
public class BankOverrideDemo {
    public static void main(String[] args) {
        // Superclass references pointing to different subclass objects
        BankAccount[] accounts = {
            new SavingsAccount("SA101", "Rahul", 10000),
            new FixedDepositAccount("FD201", "Priya", 50000, 5),
            new CurrentAccount("CA301", "Vikram", 20000)
        };
        for (BankAccount acc : accounts) {
            System.out.println("==== " + acc.getClass().getSimpleName() + " =====");
        }
    }
}
```

```

        acc.displayDetails();
        acc.applyInterest(); // dynamic dispatch
        System.out.println("----- After Interest -----");
        acc.displayDetails();
        System.out.println();
    }
}
}

```

■ Compilation & Execution Commands

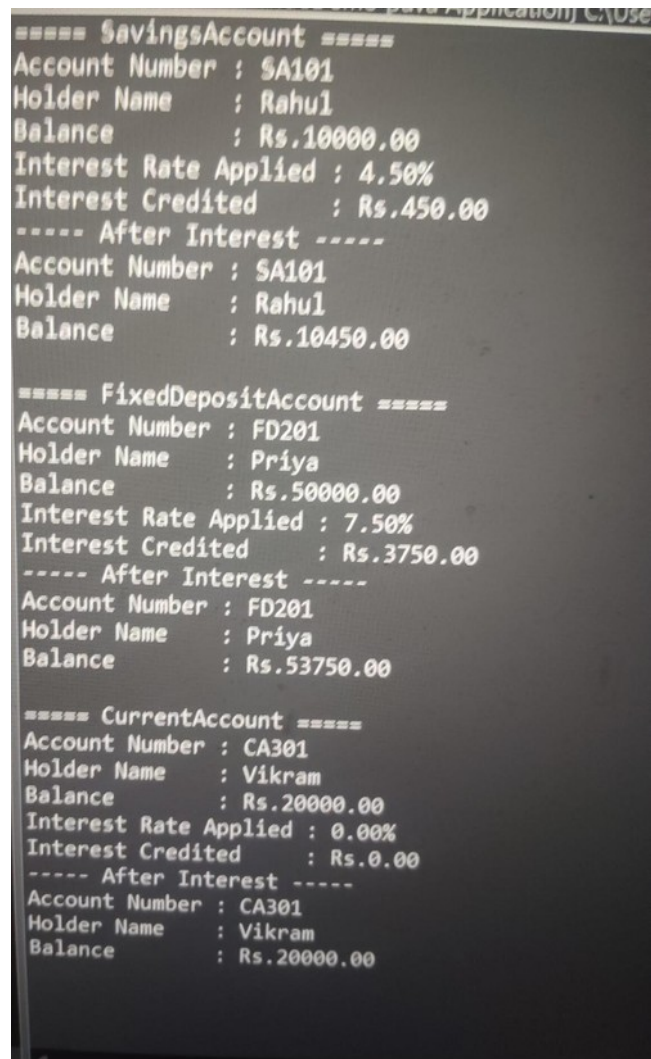
Save the source code exactly as: **BankOverrideDemo.java**

Compile: `javac BankOverrideDemo.java`

Run: `java BankOverrideDemo`

■ Output Screen

Actual console output (screenshot from execution):



```

===== SavingsAccount =====
Account Number : SA101
Holder Name    : Rahul
Balance       : Rs.10000.00
Interest Rate Applied : 4.50%
Interest Credited : Rs.450.00
----- After Interest -----
Account Number : SA101
Holder Name    : Rahul
Balance       : Rs.10450.00

===== FixedDepositAccount =====
Account Number : FD201
Holder Name    : Priya
Balance       : Rs.50000.00
Interest Rate Applied : 7.50%
Interest Credited : Rs.3750.00
----- After Interest -----
Account Number : FD201
Holder Name    : Priya
Balance       : Rs.53750.00

===== CurrentAccount =====
Account Number : CA301
Holder Name    : Vikram
Balance       : Rs.20000.00
Interest Rate Applied : 0.00%
Interest Credited : Rs.0.00
----- After Interest -----
Account Number : CA301
Holder Name    : Vikram
Balance       : Rs.20000.00

```

■ Result

The Bank application was successfully implemented in Java, demonstrating method overriding and dynamic polymorphism. Superclass BankAccount references were used to invoke calculateInterestRate(), and the JVM correctly resolved each call to the subclass-specific overridden version at runtime, confirming dynamic method dispatch.